Compte-rendu Virtualisation

Déploiement d'une Architecture Client-Serveur sous Debian

Module : Infrastructure B2 **Établissement :** EPSI **Auteur :** Rahantanirina Jordy

Sommaire du projet

- **1. Introduction :** Contexte et objectifs de l'architecture.
- **2. Configuration réseau :** Paramétrage NAT des machines virtuelles.
- **3. Sécurisation :** Mise en place des clés SSH (Ed25519).
- **4. Serveur Back-End :** API sur le port 8080 (Apache/PHP).
- **5. Serveur Front-End :** Interface utilisateur sur le port 80.
- **6. Validation :** Tests de communication inter-serveurs (JSON).
- **7. Conclusion :** Bilan des compétences acquises.

1. Introduction et contexte du projet

Dans le cadre du module d'infrastructure de la deuxième année de Bachelor (B2) à l'EPSI, ce projet vise à concevoir et déployer une architecture client-serveur fonctionnelle et sécurisée. L'objectif est de configurer deux machines virtuelles sous Debian interconnectées en réseau local. Le serveur Front-End a pour rôle d'héberger l'interface utilisateur sur le port standard 80, tandis que le serveur Back-End expose une API sur le port 8080 chargée de stocker des fichiers et de renvoyer les métadonnées au format JSON. Ce document détaille les phases d'administration système, de la sécurisation des échanges via SSH jusqu'au paramétrage avancé des services web Apache et PHP permettant la communication entre les deux nœuds.

2. Configuration Réseau et Système

Les deux machines virtuelles Debian ont été déployées sur un réseau NAT sous VMware afin de garantir leur interconnexion locale tout en conservant un accès à Internet pour l'installation des paquets. Le serveur Front-End s'est vu attribuer l'adresse IP **192.168.93.133**, tandis que le serveur Back-End utilise l'adresse IP **192.168.93.134**.

3. Sécurisation des Accès (Jalon J2 - SSH)

L'administration de l'infrastructure a été sécurisée en remplaçant l'authentification par mot de passe par une authentification par clés asymétriques. Une paire de clés cryptographiques a été générée via la commande `ssh-keygen -t ed25519`. La clé publique a ensuite été intégrée dans le fichier `~/.ssh/authorized_keys` de chaque serveur. Cette implémentation garantit un accès distant hautement sécurisé et facilite les transferts de fichiers de développement (tels que les scripts PHP) à l'aide de la commande `scp` sans nécessiter de mot de passe interactif.

> **Bonne pratique de sécurité :** L'algorithme **Ed25519** a été privilégié par rapport au classique RSA, car il offre une meilleure robustesse cryptographique tout en générant des clés beaucoup plus courtes et rapides à traiter par les serveurs.

4. Configuration du Serveur Back-End (Port 8080)

Le serveur Back-End a été paramétré pour fonctionner comme une API de stockage et de consultation. Le service web Apache2 a été reconfiguré (via la modification des fichiers `ports.conf` et `000-default.conf`) pour écouter exclusivement sur le port 8080. L'environnement a été complété par l'installation du module PHP 8.4.

- **Gestion des permissions :** Le répertoire cible `/data/uploads` a été configuré avec les droits système appropriés (`chown www-data:www-data` et `chmod 755`) pour autoriser le processus web à y enregistrer les fichiers uploadés.
- **Routage Apache :** Afin que le script intercepte correctement les requêtes API (telles que `/api/fichiers`), la directive `FallbackResource /index.php` a été ajoutée au bloc Directory du VirtualHost. Cette règle redirige toutes les routes non trouvées vers le script principal, assurant ainsi le bon fonctionnement du format JSON renvoyé au client.



5. Déploiement du Serveur Front-End (Port 80)

L'interface utilisateur a été déployée sur la machine `debian-front`. Le service Apache2 a été configuré pour écouter sur le port standard HTTP (80), accompagné du module `libapache2-mod-php` pour le traitement dynamique des pages. L'application cliente, contenue dans le fichier `index.php`, a été placée à la racine web `/var/www/html/`. Ce script a été configuré pour cibler explicitement l'adresse IP et le port du serveur Back-End (`http://192.168.93.134:8080`), permettant d'interroger l'API distante à l'aide des fonctions de flux natives de PHP (`file_get_contents` et `stream_context_create`).

6. Tests et Validation de la Communication

La validation de l'architecture a été confirmée en accédant à l'interface via un navigateur client côté Windows. Le Front-End exécute avec succès ses requêtes HTTP vers le Back-End, récupère les métadonnées au format JSON et les désérialise pour générer dynamiquement le tableau d'affichage HTML. Le routage mis en place côté serveur gère correctement les accès aux points de terminaison `/api/fichiers` et `/api/info`. La remontée des informations valide la parfaite interconnexion réseau des deux machines.

```
// Structure du payload JSON retourné par l'API Back-End (Route /api/fichiers)

       [

         {

           "nom": "fichier_test.txt",

           "taille": 930, // converti en "octets" sur le front

           "date": "2026-10-23 15:55:05"

         }

       ]
```

7. Conclusion

La réalisation de ce projet d'infrastructure a permis de consolider les compétences fondamentales en administration système sous Linux. La séparation des rôles entre les serveurs Front-End et Back-End a concrétisé la mise en pratique de concepts clés : la sécurisation des accès par clés cryptographiques SSH, la gestion avancée des permissions systèmes en ligne de commande, la configuration des VirtualHosts Apache, et l'établissement d'une communication inter-serveurs fonctionnelle via la consommation d'une API. L'environnement produit est robuste et répond au cahier des charges de l'évaluation.